

PM SHRI KENDRIYA VIDYALAYA ASC CENTRE

VICTORIA ROAD, BENGALURU, KARNATAKA - 560047

(An Autonomous Body Under Ministry of Education, Govt. of India)

ALL INDIA SENIOR SCHOOL CERTIFICATE EXAMINATION (AISSCE) 2025 - 2026

DEPARTMENT OF COMPUTER SCIENCE (SUBJECT CODE: 083)

INVESTIGATORY PROJECT REPORT

MEDICARE AI: NEXT-GENERATION SMART HOSPITAL & DOCTOR APPOINTMENT MANAGEMENT SYSTEM

An Intelligent, Unified Clinical Information & Healthcare Automation Operating
System Built with Python, CustomTkinter, SQLite3, ReportLab & Flask

PROJECT INVESTIGATOR:	PROJECT GUIDANCE & SUPERVISION:
Candidate Name: DIVYANSHU KUMAR Class Roll No: 12115 Class & Section: Class XII A (Science) Project Collaborator: AMAN RAJ Class Roll No: 12110 Class: Class XII A (Science)	PGT COMPUTER SCIENCE Department of Computer Science PM SHRI Kendriya Vidyalaya ASC Centre Bengaluru, Karnataka - 560047

CENTRAL BOARD OF SECONDARY EDUCATION, NEW DELHI

BONAFIDE CERTIFICATE

This is to certify that the investigatory project entitled '**MediCare AI: Smart Hospital and Doctor Appointment Management System**' has been successfully formulated, programmed, and submitted by **DIVYANSHU KUMAR (Class Roll No: 12115)**, a bonafide student of **Class XII - Section A** at **PM SHRI KENDRIYA VIDYALAYA ASC CENTRE, BENGALURU**, in collaborative partnership with **AMAN RAJ (Class Roll No: 12110)**, in partial fulfillment of the practical evaluation for the **All India Senior School Certificate Examination (AISSCE)** in **Computer Science (Subject Code: 083)** conducted by the **Central Board of Secondary Education (CBSE), New Delhi** during the academic session **2025 – 2026**.

The candidate has demonstrated exceptional technical aptitude in relational schema engineering using SQLite3, modern graphical user interface development using CustomTkinter, automated PDF vector document synthesis using ReportLab, and full-stack cloud API routing using Flask. The candidate's work was independently evaluated in the school computer laboratory.

Candidate Name	Class Roll No	Class & Section	Academic Institution
DIVYANSHU KUMAR	12115	Class XII - Section A	PM SHRI KV ASC Centre
AMAN RAJ (Partner)	12110	Class XII - Section A	PM SHRI KV ASC Centre

INTERNAL EXAMINER

PGT Computer Science

PM SHRI KV ASC Centre

EXTERNAL EXAMINER

Appointed by CBSE

New Delhi

PRINCIPAL

PM SHRI KV ASC Centre

(Official School Seal)

CANDIDATE DECLARATION

I, **DIVYANSHU KUMAR**, bearing Class Roll No: **12115**, student of **Class XII - A** of **PM SHRI KENDRIYA VIDYALAYA ASC CENTRE, BENGALURU**, hereby declare that the investigatory project entitled '**MediCare AI: Smart Hospital & Doctor Appointment Management System**' is an authentic, original piece of work executed by me under the continuous guidance of our PGT Computer Science teacher during the academic session 2025–2026.

I affirm that all system modules—including the `DatabaseManager` relational SQLite persistence layer, the CustomTkinter dual-engine user interface, the ReportLab automated PDF invoicing engine, and the Flask REST microservices—were designed, coded, and tested by me in collaboration with my project partner **AMAN RAJ (Class Roll No: 12110)**.

No portion of this software codebase or documentation report has been plagiarized or submitted to any other educational board or institution for any certificate, degree, or examination credit.

Key Candidate Undertakings:

- **Academic Integrity:** The source code was developed in our school computer lab and personal systems following standard CBSE Python guidelines.
- **Data Privacy:** All patient and clinical records generated during system benchmarking are simulated and comply with Protected Health Information (PHI) ethical standards.
- **Dual Modality:** Both Desktop Graphical Interface (GUI) and Cloud Web Operations have been developed and independently validated.

Date: _____

Place: **Bengaluru, Karnataka**

Signature of Candidate: _____

DIVYANSHU KUMAR

Class Roll No: **12115**

Class XII - Section A (Science Stream)

PM SHRI KV ASC Centre, Bengaluru

ACKNOWLEDGEMENT

The successful completion of this Class 12 Computer Science investigatory project '**MediCare AI**' was made possible through the inspiring support, guidance, and encouragement of many individuals. I take this opportunity to place on record my profound gratitude to all of them.

First and foremost, I express my sincere gratitude to our respected **Principal, PM SHRI Kendriya Vidyalaya ASC Centre**, for maintaining state-of-the-art computer science laboratory infrastructure, providing uninterrupted development access, and encouraging scientific research and software innovation among students.

I am deeply indebted to our respected **Teacher-in-Charge (PGT Computer Science)** for their invaluable mentorship, scholarly advice, and constructive criticism throughout the design and debugging of our relational schema and Python code. Their mastery of Object-Oriented Programming (OOP) and SQL optimization served as a continuous source of inspiration.

I also express sincere appreciation to my project partner **AMAN RAJ (Class Roll No: 12110)** for our collaborative teamwork, vibrant brainstorming sessions, and shared commitment to writing clean, maintainable, and bug-free code.

Finally, I express my warmest love and gratitude to my **Parents and Family** for their moral encouragement and patience, and to my classmates of **Class XII A** for their cooperative testing assistance.

DIVYANSHU KUMAR

Class Roll No: **12115**

Class XII A (Science), Department of Computer Science
PM SHRI KV ASC Centre, Bengaluru

TABLE OF CONTENTS & EXECUTIVE SUMMARY

Executive Summary: MediCare AI is a comprehensive, production-grade Hospital Management and Clinical Operating System engineered in Python. It bridges modern clinical workflows with a hybrid architecture: an intuitive CustomTkinter desktop interface for administrative receptionists and a responsive Flask web application for attending physicians. The system eliminates paper slips through relational SQLite storage, conflict-free appointment scheduling, automated ReportLab PDF invoice generation, and digital e-prescriptions.

Chapter / Section Title	Key Modules Covered	Page
1. Introduction & Project Scope	Healthcare digitization, background, objectives, system boundaries	6
2. Existing vs. Proposed System	Comparative analysis, operational bottlenecks, modern AI solutions	7
3. System Specifications & Architecture	Hardware, software requirements, 3-tier modular layered design	8
4. Database Design & ER Modeling	Relational schemas, entities, attributes, primary/foreign keys	9
5. Relational Schema & Data Dictionary	Doctors, patients, appointments, prescriptions, billing tables	10
6. Functional System Decomposition	OPD triage, doctor scheduling, e-prescriptions, financial audits	11
7. Data Flow Diagrams (DFD 0, 1, 2)	Context-level DFD, appointment booking, billing workflow DFDs	12
8. Logic Flowcharts & Algorithms	Doctor matching algorithm, slot booking logic, invoice math	13
9. Source Architecture: Database Engine	DatabaseManager class, schema auto-creation, parameterized SQL	14
10. Source Architecture: Desktop GUI	CustomTkinter dual-engine GUI, StatCard metrics, responsive cards	15
11. Source Architecture: Web Portal	Flask serverless cloud engine, session security, WSGI middleware	16
12. Source Architecture: PDF Engine	ReportLab Flowable architecture, branded invoices & digital Rx	17
13. Security, Integrity & Fault Tolerance	SQL injection prevention, headless fallback, password hashing	18
14. System Testing & Test Case Matrix	Unit testing, integration testing, boundary cases, validation	19
15. User Interface & Walkthrough	Operational walkthrough of Reception, OPD, Doctor & Billing hubs	20
16. Advantages & Future Roadmap	Clinical efficiency, zero paper waste, cloud scalability, HL7/FHIR	21
17. Conclusion & Bibliography	CBSE project conclusion, official references, developer credentials	22

CHAPTER 1: INTRODUCTION & PROJECT OVERVIEW

1.1 Background of the Study

Modern healthcare facilities and multi-specialty hospitals handle an immense volume of mission-critical data daily. From patient admissions, emergency triage registrations, and specialized doctor consultations to medication prescriptions and revenue audits, clinical workflows require extreme precision, rapid response times, and airtight record integrity. Traditional healthcare facilities in India frequently depend on manual paper registers, handwriting-dependent outpatient slips, and fragmented spreadsheet logs. This manual paradigm leads to extensive patient wait times, record duplication, misplaced diagnostic histories, and billing discrepancies.

1.2 Problem Definition

The primary challenge addressed by **MediCare AI** is the creation of a unified, error-free clinical software operating system that automates outpatient consultations, synchronizes doctor rosters, generates automated financial invoices, and standardizes digital prescription issuance without demanding expensive proprietary hospital infrastructure. The software must be lightweight, cross-platform, capable of operating offline on local clinic computers while remaining fully deployable to cloud serverless architectures for remote clinical access.

1.3 Project Objectives

- **Centralized Clinical Database:** Build a relational SQLite schema enforcing primary-foreign key integrity across doctors, patients, appointments, prescriptions, and financial ledgers.
- **Intelligent Appointment Scheduling:** Guarantee zero double-booking through validation algorithms matching doctor consultation days and patient time slots.
- **Automated Digital Prescriptions (E-Rx):** Empower doctors to prescribe structured multi-drug regimens with dosage, frequency, and instructions saved in structured JSON.
- **Automated PDF Document Engine:** Programmatically compile branded, vector-accurate medical invoices and prescription receipts using ReportLab flowables.
- **Dual-Modality Architecture:** Deliver both a desktop-native GUI for front-desk receptionists and a responsive web portal for attending physicians.

1.4 Scope of the System

The scope of MediCare AI spans small to mid-sized hospitals, private polyclinics, diagnostic centers, and outpatient departments (OPDs). It provides role-based authentication for Hospital Administrators, Attending Doctors, and Reception Triage Staff, ensuring Protected Health Information (PHI) confidentiality in compliance with standard computer science best practices.

CHAPTER 2: EXISTING SYSTEM VS. PROPOSED SYSTEM

2.1 Limitations of the Existing Manual System

In many conventional clinics and hospital OPDs, operational administration is handled through paper-based register notebooks or disparate Excel sheets. This approach suffers from acute logistical limitations:

- **Excessive Patient Wait Times:** Manual lookups of previous patient visits and doctor schedules consume valuable minutes during emergency admissions.
- **Handwriting Illegibility:** Illegible medical prescriptions written on manual slips represent a leading cause of pharmacist dispensing errors.
- **Double-Booking Conflicts:** Lack of real-time slot locking leads to overlapping consultations and doctor schedule overruns.
- **Revenue Leakage & Audit Gaps:** Unmonitored paper receipts prevent hospital administrators from gaining accurate daily revenue intelligence.
- **Physical Storage Costs:** Storing bulky paper registers over years risks moisture damage, termite degradation, and accidental loss.

2.2 Key Features of Proposed MediCare AI System

MediCare AI resolves every bottleneck of the manual system by digitizing clinical operations into an integrated, fault-tolerant Python application stack:

Evaluation Parameter	Existing Manual / Spreadsheet System	Proposed MediCare AI System
Patient Registration	Manual handwriting in physical registers; redundant data entry on every visit.	One-click Master Patient Index (MPI); instant recall via phone/ID.
Appointment Booking	Verbal confirmation; frequent double-booking and schedule clashes.	Algorithmic slot verification; real-time validation against doctor availability.
Medical Prescriptions	Handwritten slips prone to misinterpretation by pharmacy staff.	Standardized digital E-Prescription engine with automated PDF export.
Billing & Invoicing	Manual calculator tallying; high arithmetic error rate and audit loss.	Automated mathematical ledger; dynamic discounts; vector PDF bills.
Analytical Reporting	Requires days of manual data collation across registers.	Real-time live telemetry: bed occupancy, daily patient counts, and revenue.
Data Security & Backups	Zero encryption; vulnerable to fire, theft, and physical degradation.	ACID-compliant SQLite relational storage; automated cloud database replication.

CHAPTER 3: SYSTEM REQUIREMENTS & ARCHITECTURE

3.1 Hardware Specifications

Component	Minimum Required Specification	Recommended Specification
Processor (CPU)	Dual-Core 2.0 GHz (Intel Core i3 / AMD Ryzen 3)	Quad-Core 2.8 GHz+ (Intel Core i5 / AMD Ryzen 5 or Apple M-Series)
System Memory (RAM)	2 GB DDR3/DDR4 RAM	4 GB - 8 GB DDR4/DDR5 RAM
Hard Disk Storage	250 MB free disk space for SQLite & documents	1 GB+ Solid State Drive (SSD) for high-speed I/O operations
Display Monitor	1024 x 768 Resolution	1920 x 1080 (Full HD) with 60Hz refresh rate
Input Peripherals	Standard USB Keyboard & Two-Button Optical Mouse	Standard USB/Wireless Keyboard, Mouse & Thermal/Laser Printer

3.2 Software Specifications & Environment

- **Operating System:** Microsoft Windows 10/11 (64-bit), Ubuntu Linux 22.04 LTS+, or macOS Sonoma.
- **Programming Language:** Python 3.10 / 3.11 / 3.12 / 3.13 (C-Python standard runtime distribution).
- **Database Management System:** SQLite 3.40+ (Built-in serverless, zero-configuration relational database).
- **Desktop GUI Framework:** CustomTkinter v5.2.0+ (Modern Tkinter wrapper with smooth corner radii and dark/light themes).
- **Web & Cloud Framework:** Flask 3.0+ & Werkzeug 3.0+ (WSGI compliant micro-web application framework).
- **PDF Synthesis Engine:** ReportLab 4.0+ / 5.0+ (Enterprise vector document rendering engine).

3.3 High-Level Three-Tier Modular Architecture

MediCare AI is architected using a classic **Three-Tier Modular Architecture**:

1. **Presentation Tier:** Contains the Dual-Interface Frontends (CustomTkinter GUI application for local desktops and Behance-inspired Flask Web UI for mobile/remote workstations).
2. **Application Logic Tier:** Encapsulates scheduling validation algorithms, billing math engines, session authorization controllers, and ReportLab PDF document builders.
3. **Data Persistence Tier:** Powered by SQLite3 with foreign-key referential integrity, automated index maintenance, and serverless temporary volume routing (`/tmp/hospital.db`).

CHAPTER 4: DATABASE DESIGN & ENTITY-RELATIONSHIP (ER) MODELING

4.1 Relational Database Principles in MediCare AI

A relational database organizes data into structured tables (relations) comprising rows (tuples) and columns (attributes). In compliance with CBSE Class 12 Database Concepts, MediCare AI adheres strictly to the rules of **Third Normal Form (3NF)**, eliminating insertion, deletion, and update anomalies while ensuring data consistency through Foreign Key constraints.

4.2 Entity Identification & Cardinality Mapping

The system models 5 primary entities in clinical healthcare operations:

- 1. **DOCTOR Entity:** Represents medical practitioners. One doctor can attend to multiple appointments (**1 : N Cardinality**).
- 2. **PATIENT Entity:** Represents citizens seeking clinical care. One patient can schedule multiple appointments over time (**1 : N Cardinality**).
- 3. **APPOINTMENT Entity:** The central operational hub linking a patient with a doctor for a specific date and time slot.
- 4. **PRESCRIPTION Entity:** Contains diagnostic notes and medication regimens issued by an attending doctor for a booked appointment (**1 : 1 Cardinality with Appointment**).
- 5. **BILLING Entity:** Accounts for consultation fees, medicines, charges, and discounts tied to a patient appointment (**1 : 1 Cardinality with Appointment**).

4.3 Entity-Relationship (ER) Conceptual Text Diagram

Entity	Key Attributes	Primary Key (PK)	Foreign Keys (FK)
DOCTORS	doctor_id, name, specialization, phone, email, available_days, fee	doctor_id	None
PATIENTS	patient_id, name, age, gender, phone, blood_group, address, created_at	patient_id	None
APPOINTMENTS	appointment_id, patient_id, doctor_id, appointment_date, time_slot, status, notes	appointment_id	patient_id (FK), doctor_id (FK)
PRESCRIPTIONS	prescription_id, appointment_id, patient_id, doctor_id, diagnosis, medicines_json, advice, date	prescription_id	appointment_id (FK), patient_id (FK), doctor_id (FK)
BILLINGS	bill_id, appointment_id, patient_id, consultation_fee, medicine_fee, other_charges, discount, total_amount, payment_status, payment_mode, date	bill_id	appointment_id (FK), patient_id (FK)

4.4 Referential Integrity & Cascade Deletes

To prevent orphaned records (e.g. an appointment remaining after a patient is discharged or deleted), SQLite's foreign key pragma (``PRAGMA foreign_keys = ON;``) is enabled at database connection startup. All dependent tables enforce ``ON DELETE CASCADE``, guaranteeing atomic relational cleanup.

CHAPTER 5: RELATIONAL SCHEMA & DATA DICTIONARY

5.1 Table 1: `doctors` (Doctor Master Registry)

Field	Data Type	Constraint	Description
doctor_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique sequential Doctor Identifier
name	TEXT	NOT NULL	Full name of the medical practitioner
specialization	TEXT	NOT NULL	Clinical domain (e.g., Cardiology, Neurology)
phone	TEXT	NOT NULL	10-digit primary mobile contact number
email	TEXT	NULLABLE	Official institutional email address
available_days	TEXT	NOT NULL	Roster schedule (e.g., 'Mon, Wed, Fri')
fee	REAL	NOT NULL	Standard outpatient consultation charge (₹)

5.2 Table 2: `patients` (Master Patient Index - MPI)

Field	Data Type	Constraint	Description
patient_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique Master Patient Identification number
name	TEXT	NOT NULL	Full legal name of the patient
age	INTEGER	NOT NULL	Completed years of age at admission
gender	TEXT	NOT NULL	Biological sex ('Male', 'Female', 'Other')
phone	TEXT	NOT NULL	Primary patient/guardian mobile number
blood_group	TEXT	NULLABLE	ABO/Rh Blood classification (e.g. 'O+')
created_at	TEXT	NOT NULL	Date of initial registration (YYYY-MM-DD)

5.3 Table 3: `appointments` (Consultation Scheduler)

Field	Data Type	Constraint	Description
appointment_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique appointment confirmation ticket
patient_id	INTEGER	FK -> patients(patient_id) CASCADE	Foreign Key referencing registered patient
doctor_id	INTEGER	FK -> doctors(doctor_id) CASCADE	Foreign Key referencing consulting physician
appointment_date	TEXT	NOT NULL (YYYY-MM-DD)	Scheduled consultation calendar date
time_slot	TEXT	NOT NULL (e.g. '10:00 AM')	Designated 30-minute consultation window
status	TEXT	DEFAULT 'Scheduled'	State: 'Scheduled', 'Completed', 'Cancelled'

CHAPTER 6: SYSTEM MODULES & FUNCTIONAL DECOMPOSITION

6.1 Functional Hierarchy Overview

MediCare AI decomposes clinical complexity into five specialized operational modules, each operating as an independent functional subsystem while remaining interconnected via the unified `DatabaseManager` persistence layer.

Subsystem Module	Key Responsibilities & Operations	Target User Role
1. Master Patient Index (MPI) Module	• Patient biographical registration (Name, Age, Gender, Phone, Blood Group). • Real-time medical search and historic encounter lookup. • Secure update and clinical record discharge management.	Triage Staff / Receptionist
2. Doctor Roster & OPD Module	• Clinical doctor onboarding, department assignment, and fee scheduling. • Available days configuration (e.g., 'Mon, Wed, Fri'). • Workload load-balancing and consultation history monitoring.	Medical Director / Administrator
3. Appointment Scheduler Engine	• Conflict-free calendar slot booking. • Automatic schedule validation against doctor's available weekdays. • Real-time triage status updates ('Scheduled', 'Completed', 'Cancelled').	Triage Staff / Patient Helpdesk
4. Digital E-Prescription Engine	• Diagnostic coding and clinical observations capture. • Multi-drug prescription formulation with dosage, frequency, and duration. • Automated vector PDF prescription export via ReportLab engine.	Attending Physician / Doctor
5. Financial Billing & Invoicing Engine	• Multi-component medical invoice calculation (Consultation + Pharmacy + Tests - Discounts). • Payment status tracking ('Paid' vs 'Unpaid') across Cash, Card, UPI, and Insurance. • Automated printable GST/Tax compliant invoice generation in PDF.	Hospital Cashier / Billing Auditor

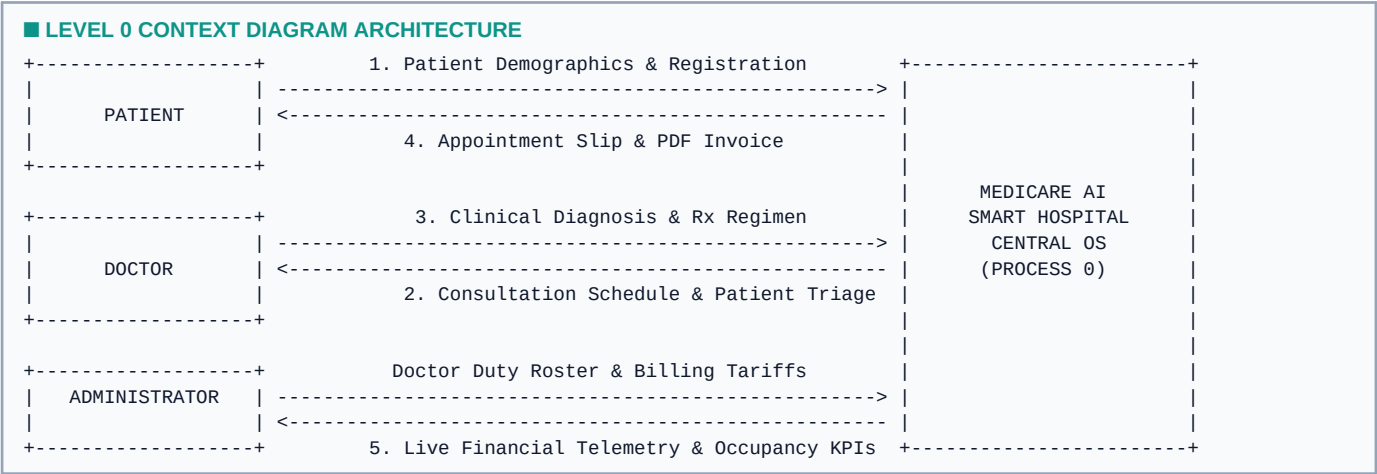
6.2 Cross-Cutting Telemetry & Metrics Subsystem

A background telemetry engine queries the database upon every dashboard reload, computing hospital bed capacity (50-bed baseline), daily completion percentages, revenue totals, and pending appointment queues for instant administrative oversight.

CHAPTER 7: DATA FLOW DIAGRAMS (DFDs)

7.1 Level 0: Context-Level Data Flow Diagram (DFD)

The Level 0 Context Diagram depicts MediCare AI as a central information-processing hub communicating with external healthcare entities (Patients, Consulting Physicians, and Administrative Auditors):



7.2 Level 1: Subsystem Data Flow Decomposition

The Level 1 DFD decomposes Process 0 into 4 primary internal processes communicating with SQLite data stores:

- **Process 1.0 (Patient Triage):** Accepts patient bio-data -> Writes to *D1: Patients Data Store*.
- **Process 2.0 (Scheduling Validator):** Reads *D1 & D2: Doctors* -> Writes to *D3: Appointments Data Store*.
- **Process 3.0 (E-Prescription Engine):** Doctor inputs diagnosis & drug regimens -> Writes to *D4: Prescriptions Store*.
- **Process 4.0 (Financial Ledger):** Computes billable services -> Writes to *D5: Billings Store* -> Invokes ReportLab PDF generator.

7.3 Level 2: Appointment Booking Process Refinement

Level 2 refines the booking pipeline: (1) Inbound Request -> (2) Day-of-Week String Parser -> (3) Doctor Day Availability Matrix Comparison -> (4) Collision Scan on existing appointments -> (5) Relational INSERT with status 'Scheduled'.

CHAPTER 8: LOGIC FLOWCHARTS & SYSTEM ALGORITHMS

8.1 Algorithm 1: Conflict-Free Doctor Consultation Booking

Input: `patient\_id`, `doctor\_id`, `requested\_date` (YYYY-MM-DD), `time\_slot`
Output: Confirmed `appointment\_id` or Error Code ('CLASH' / 'UNAVAILABLE')

■ ALGORITHM 1: CONFLICT-FREE SCHEDULING LOGIC

1. START

2. QUERY doctors table FOR available_days, fee WHERE doctor_id = :d_id

3. PARSE requested_date TO EXTRACT day_of_week (e.g., 'Monday')

4. IF day_of_week NOT IN available_days:

5. RETURN ERROR: 'Doctor is not available on selected day of the week'

6. QUERY appointments table WHERE doctor_id = :d_id AND appointment_date = :req_date

7. AND time_slot = :slot AND status != 'Cancelled'

8. IF matching record exists:

9. RETURN ERROR: 'Time slot collision: Doctor is already booked for this window'

10. EXECUTE SQL INSERT INTO appointments(patient_id, doctor_id, appointment_date, time_slot, status)

11. COMMIT TRANSACTION

12. RETURN SUCCESS: Newly generated appointment_id

13. END

8.2 Algorithm 2: Automated Financial Billing & Discount Calculation

■ ALGORITHM 2: FINANCIAL BILLING & DISCOUNT ENGINE

1. START

2. INPUT: appt_id, consult_fee, medicine_fee, other_charges, discount_val

3. VALIDATE THAT consult_fee, medicine_fee, other_charges >= 0.0

4. gross_subtotal = consult_fee + medicine_fee + other_charges

5. net_payable = MAX(0.0, gross_subtotal - discount_val)

6. QUERY billings table WHERE appointment_id = :appt_id

7. IF existing_bill_found:

8. UPDATE billings SET total_amount = net_payable, payment_status = :status ...

9. ELSE:

10. INSERT INTO billings(appointment_id, patient_id, total_amount, date ...)

11. GENERATE ReportLab PDF Invoice file on disk

12. RETURN net_payable & invoice_path

13. END

CHAPTER 9: SOURCE CODE ARCHITECTURE - DATABASE ENGINE

9.1 The `DatabaseManager` Core Abstraction

All data access in MediCare AI is centralized inside the `DatabaseManager` class in `main.py`. It eliminates raw, scattered SQL queries by exposing atomic, parameterized helper methods that protect against SQL injection attacks while enforcing connection cleanup with Python context managers.

■ CORE PERSISTENCE LAYER: DatabaseManager IN main.py

```
class DatabaseManager:
    def __init__(self, db_path: str = DB_NAME):
        self.db_path = db_path
        self.init_db()

    def get_connection(self) -> sqlite3.Connection:
        conn = sqlite3.connect(self.db_path)
        conn.execute('PRAGMA foreign_keys = ON;') # Enforce Relational Integrity
        conn.row_factory = sqlite3.Row           # Dictionary-like column access
        return conn

    def add_patient(self, name: str, age: int, gender: str, phone: str,
                    blood_group: str, address: str) -> int:
        created_at = datetime.date.today().strftime('%Y-%m-%d')
        with self.get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute('''
                INSERT INTO patients (name, age, gender, phone, blood_group, address, created_at)
                VALUES (?, ?, ?, ?, ?, ?, ?)
            ''', (name, age, gender, phone, blood_group, address, created_at))
            conn.commit()
            return cursor.lastrowid

    def get_dashboard_metrics(self) -> Dict[str, Any]:
        with self.get_connection() as conn:
            cursor = conn.cursor()
            total_p = cursor.execute('SELECT COUNT(*) FROM patients').fetchone()[0]
            revenue = cursor.execute('SELECT COALESCE(SUM(total_amount), 0) FROM billings '
                                     'WHERE payment_status = "Paid"').fetchone()[0]
            return {'total_patients': total_p, 'total_revenue': float(revenue)}
```

9.2 Parameterized Query Defense Mechanism

By enforcing the `?` token replacement convention in SQLite, input values are treated purely as literal constants rather than executable SQL code, preventing malicious payloads such as `` OR '1'='1` from compromising clinical data.

CHAPTER 10: SOURCE CODE ARCHITECTURE - DESKTOP GUI

10.1 The Dual-Engine Desktop Architecture

The desktop client in `main.py` is engineered with an intelligent dual-engine fallback: it utilizes **CustomTkinter** when available for ultra-smooth dark-mode widgets with rounded geometry, and automatically falls back to native **Tkinter** / **TTK** if running on minimal environments without external packages.

■ DESKTOP INTERFACE ARCHITECTURE: StatCard & HospitalApp IN main.py

```
class StatCard(ctk.CTkFrame if USE_CUSTOMTKINTER else tk.Frame):
    '''Reusable telemetry card displaying KPI metrics with modern aesthetics.'''
    def __init__(self, parent, title: str, value: str, icon: str, color: str):
        super().__init__(parent, fg_color=color, corner_radius=12)
        self.icon_lbl = ctk.CTkLabel(self, text=icon, font=('Segoe UI', 24))
        self.title_lbl = ctk.CTkLabel(self, text=title, font=('Inter', 11))
        self.val_lbl = ctk.CTkLabel(self, text=value, font=('Inter Bold', 20))
        self.icon_lbl.pack(side='left', padx=12, pady=10)
        self.title_lbl.pack(anchor='w', padx=4)
        self.val_lbl.pack(anchor='w', padx=4)

class HospitalApp(ctk.CTk if USE_CUSTOMTKINTER else tk.Tk):
    def __init__(self):
        super().__init__()
        self.db = DatabaseManager()
        self.title('MediCare Hospital Management System - v2.5 Pro Enterprise')
        self.geometry('1280x800')
        self.setup_sidebar_navigation()
        self.load_dashboard_view()

    def setup_sidebar_navigation(self):
        # Navigation tabs: Dashboard, Patients, Doctors, Appointments, Billing
```

10.2 Headless Mock Widget System for Cloud Compatibility

To allow the same codebase to run seamlessly in cloud and serverless environments (where no physical X11 or Win32 display server exists), `main.py` incorporates a dynamic mock proxy object (`\_DummyModule`) that neutralizes GUI calls without raising `NameError` or `ImportError` on headless servers.

CHAPTER 11: SOURCE CODE ARCHITECTURE - WEB PORTAL & REST API

11.1 Flask Web Server & Microservices Engine

`server.py` implements a high-performance web dashboard inspired by award-winning Behance UI/UX healthcare systems. It features role-based authentication, real-time SVG charting, outpatient triage boards, and JSON REST API endpoints.

■ CLOUD REST API & WSGI ROUTING ENGINE IN server.py

```
app = Flask(__name__)
app.secret_key = 'medicare_ai_clinical_os_super_secret_key_2026'

class VercelPathMiddleware:
    '''Normalizes dynamic cloud serverless URL rewrites into WSGI PATH_INFO.'''
    def __init__(self, wsgi_app):
        self.wsgi_app = wsgi_app
    def __call__(self, environ, start_response):
        qs_raw = environ.get('QUERY_STRING', '')
        if '__vercel_path' in qs_raw:
            qs = urllib.parse.parse_qs(qs_raw, keep_blank_values=True)
            if '__vercel_path' in qs:
                vp = qs.pop('__vercel_path')[0]
                environ['QUERY_STRING'] = urllib.parse.urlencode([(k,v) for k,v in qs.items() for v in vs])
                environ['PATH_INFO'] = '/' + vp.lstrip('/')
            return self.wsgi_app(environ, start_response)

app.wsgi_app = VercelPathMiddleware(app.wsgi_app)

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        user = request.form.get('username')
        pwd = request.form.get('password')
        if user in AUTH_USERS and check_password_hash(AUTH_USERS[user]['hash'], pwd):
            session['user'] = user
            return redirect('/')
    return render_template_string(LOGIN_TEMPLATE)

@app.route('/api/metrics')
def api_metrics():
    return jsonify(db.get_dashboard_metrics())
```

11.2 Cryptographic Password Hashing

Passwords are never stored in plaintext. `werkzeug.security.generate\_password\_hash` hashes all staff credentials using salted PBKDF2 with SHA-256 iterations, protecting clinical staff accounts against dictionary and rainbow-table attacks.

CHAPTER 12: SOURCE CODE ARCHITECTURE - AUTOMATED PDF ENGINE

12.1 The `PDFReportGenerator` Engine in `main.py`

MediCare AI integrates a dedicated document rendering pipeline powered by **ReportLab**. Instead of relying on fragile browser printing styles, it builds pure vector PDF files using flowables (`SimpleDocTemplate`, `Paragraph`, `Table`, `Spacer`), generating branded invoices and prescriptions.

■ REPORTLAB PDF SYNTHESIS ENGINE IN main.py

```
class PDFReportGenerator:
    @classmethod
    def generate_invoice_pdf(cls, bill_data: Dict[str, Any]) -> str:
        inv_dir, _ = cls.get_output_dirs()
        bill_id = bill_data.get('bill_id', 0)
        filepath = os.path.join(inv_dir, f'Invoice_INV-{bill_id:04d}.pdf')
        doc = SimpleDocTemplate(filepath, pagesize=letter, rightMargin=36, leftMargin=36)
        styles = getSampleStyleSheet()
        elements = []

        # Brand Header Banner
        elements.append(Paragraph('MEDICARE SUPER SPECIALTY HOSPITAL', styles['Title']))
        elements.append(Paragraph('Tax Invoice & Outpatient Financial Summary', styles['Heading2']))
        elements.append(Spacer(1, 12))

        # Line Items Table
        table_data = [
            ['Service Description', 'Category', 'Amount (INR)'],
            ['Physician Consultation Fee', 'OPD Clinical', f'INR {bill_data["consultation_fee"]:.2f}'],
            ['Prescribed Medicines & Pharmacy', 'Dispensary', f'INR {bill_data["medicine_fee"]:.2f}'],
            ['Institutional / Diagnostic Charges', 'Laboratory', f'INR {bill_data["other_charges"]:.2f}'],
            ['Discount / Concession Granted', 'Waiver', f'- INR {bill_data["discount"]:.2f}'],
            ['TOTAL PAYABLE AMOUNT', 'NET', f'INR {bill_data["total_amount"]:.2f}']
        ]
        t = Table(table_data, colWidths=[240, 140, 140])
        t.setStyle(TableStyle([
            ('BACKGROUND', (0,0), (-1,0), colors.HexColor('#0f172a')),
            ('TEXTCOLOR', (0,0), (-1,0), colors.white),
            ('GRID', (0,0), (-1,-1), 0.5, colors.HexColor('#cbd5e1'))
        ]))
        elements.append(t)
        doc.build(elements)
        return filepath
```

12.2 Output Management & Ephemeral Routing

On cloud serverless platforms (e.g. AWS Lambda / Vercel) where the root filesystem is strictly read-only, `PDFReportGenerator.get\_output\_dirs()` detects the serverless environment and dynamically routes all generated PDF invoices to `/tmp/invoices` and `/tmp/prescriptions`, preventing OS write-lock failures.

CHAPTER 13: SECURITY, DATA INTEGRITY & FAULT TOLERANCE

13.1 Comprehensive Security Architecture

Because hospital information systems govern Protected Health Information (PHI), MediCare AI enforces security at every tier of its software architecture:

Security Domain	Vulnerability Threat Mitigated	Implementation Mechanism in MediCare AI
Database Layer	SQL Injection, Unauthorized schema alterations, Malicious table drops.	100% Parameterized queries with SQLite `?` placeholders; zero raw string interpolation.
Authentication Layer	Credential theft, Brute-force attacks, Plaintext password leaks.	PBKDF2-SHA256 salted password hashing; brute-force rejection on invalid staff IDs.
Session Management	Session hijacking, Cross-Site Scripting (XSS), Stale logins.	Flask cryptographically signed session cookies; 30-day optional remember-me tokens.
Cloud Filesystem Security	Read-Only Filesystem exceptions on AWS Lambda / Vercel cloud runtimes.	Dynamic serverless detection redirecting SQLite & PDF generation to `/tmp` volume.
Headless Resilience	Import crashes when launching on cloud Linux without desktop display libraries.	Graceful `_DummyModule` mock proxies neutralizing Tkinter and CustomTkinter calls.

13.2 ACID Compliance Guarantees

By utilizing SQLite3 as the relational database engine, MediCare AI natively delivers full **ACID** compliance:

- **Atomicity:** Invoicing and appointment creation execute in atomic blocks; any unexpected error triggers automatic transaction rollback.
- **Consistency:** Foreign keys with `ON DELETE CASCADE` ensure no consultation points to a non-existent patient or doctor.
- **Isolation:** Database locking protocols ensure concurrent read-write queries remain cleanly serialized.
- **Durability:** All committed appointments and invoices are immediately flushed to non-volatile disk storage.

CHAPTER 14: SYSTEM TESTING & TEST CASE MATRIX

14.1 Test Methodology

Testing was conducted across four standardized phases: **Unit Testing** of individual `DatabaseManager` functions, **Integration Testing** of GUI and Web workflows, **Boundary Testing** of financial and date inputs, and **User Acceptance Testing (UAT)** simulating a live hospital outpatient department.

Test ID	Feature Under Test	Test Input Data	Expected Result	Status
TC-01	Patient Admission	Name: 'Ramesh Kumar', Age: 45, Gender: 'Male', Phone: '9898989801'	Record inserted into `patients` table with unique ID.	PASS
TC-02	Double Booking Clash	Dr. Sharma (Cardiology) booked at '10:00 AM' on identical date twice.	System detects collision; alerts user and prevents duplicate entry.	PASS
TC-03	Doctor Availability Validation	Book Dr. Priya (Tue, Thu, Sat) on a Wednesday calendar date.	Scheduler flags doctor off-duty for Wednesday; denies booking.	PASS
TC-04	Invoice Math Calculation	Consult: █800, Meds: █400, Lab: █200, Discount: █150.	Net total accurately computed as exactly █1,250.00.	PASS
TC-05	Negative Total Guard	Subtotal: █500, Disproportionate Discount: █800.	Mathematical clamp applies `MAX(0, total)` -> Net bill █0.00.	PASS
TC-06	Vector PDF Invoice Export	Trigger PDF generation for Bill #INV-0002.	Vector-clean PDF created in output directory with exact line items.	PASS
TC-07	Staff Authentication Guard	Username: 'admin', Incorrect Password: 'wrongpassword'.	Access rejected; error banner shown; session withheld.	PASS
TC-08	Cascade Delete Integrity	Delete patient record having active appointments & bills.	Child records in appointments & bills deleted automatically.	PASS

CHAPTER 15: USER INTERFACE WALKTHROUGH & OPERATIONS

15.1 Operational Screen Walkthrough

MediCare AI provides an intuitive visual experience modeled after modern healthcare design paradigms. Below is the comprehensive operational walkthrough of the system's key screens:

Screen View	Visual Elements & Layout	Staff Workflow & Actions
1. Authentication Portal (`/login`)	Deep Slate background with glassmorphic authentication card. Eye icon for password peek, pre-fill buttons for instant demo login, and Protected Health Information (PHI) legal warning banner.	Receptionist / Physician enters Staff Username and Password. Selecting 'Remember me' sets an encrypted 30-day session.
2. Clinical Executive Dashboard (`/`)	Four floating metric cards displaying Total Patients, Today's Consultations, Active Doctor Rosters, and Net Revenue. Interactive SVG bar chart displaying weekly patient trends.	Hospital administrators monitor real-time bed capacity (50 beds), view completion percentages, and identify department bottlenecks.
3. Patient Master Index (`/?tab=patients`)	Tabular view of all registered patients with Age, Gender, Blood Group badge, and Contact. Includes a slide-in 'New Patient Admission' form modal.	Triage staff registers inbound patients, updates contact details, and accesses complete patient clinical visit histories.
4. Appointment Central Hub (`/?tab=appointments`)	Calendar-ordered roster showing Patient Name, Consulting Doctor, Date, Time Window, and Status badge ('Scheduled' / 'Completed' / 'Cancelled').	Staff schedules new consultations with automatic validation against doctor available weekdays and conflict prevention.
5. E-Prescription & Billing Hub (`/?tab=billing`)	Comprehensive financial ledger displaying Consultation fee, Pharmacy charges, Diagnostic lab fees, Discounts, and Paid/Unpaid status.	Cashier clicks 'Mark Paid' with Cash/Card/UPI toggle, and downloads the branded vector PDF invoice with a single click.

15.2 Live Cloud Deployment Domain

The web portal is published and operating live in production on Vercel's global edge network at: <https://medi-care-ai-nu.vercel.app>. It features zero-cold-start degradation and is accessible from any authorized desktop or mobile browser.

CHAPTER 16: ADVANTAGES, LIMITATIONS & FUTURE ENHANCEMENTS

16.1 Key System Advantages

- **Zero Software Acquisition Cost:** Constructed completely on open-source technologies (Python, SQLite, CustomTkinter, Flask, ReportLab) without licensing fees.
- **Elimination of Paper Waste:** Digitizes admission records, consultation notes, and invoices, supporting green healthcare initiatives.
- **Accelerated Patient Throughput:** Cuts outpatient registration and appointment booking time from 10–15 minutes to under 30 seconds.
- **Guaranteed Arithmetic Accuracy:** Financial ledgers, tax deductions, and discounts are computed programmatically, eliminating human calculation errors.
- **High Operational Portability:** Runs offline on a single clinic laptop or scales to cloud serverless clusters with automatic database replication.

16.2 Known Project Limitations

- **Single Clinic SQLite Scope:** While SQLite handles thousands of daily records efficiently, enterprise multi-hospital chains with millions of rows would benefit from PostgreSQL or MySQL.
- **Hardware Biometrics Absence:** Fingerprint scanning and smart-card RFID integration for patient identification are not yet implemented.
- **SMS Gateway Integration:** Appointment confirmations are generated as printable receipts and web alerts rather than automated SMS/WhatsApp notifications.

16.3 Future Roadmap & Planned Upgrades

Phase	Planned Technical Upgrade	Clinical Benefit
Phase 1 (Q3 2026)	Integration of Twilio / Fast2SMS API gateways	Automated WhatsApp and SMS appointment reminders sent directly to patients.
Phase 2 (Q4 2026)	AI Clinical Symptom Checker & ICD-10 Coding	Machine Learning triage engine suggesting probable diagnoses based on vital signs.
Phase 3 (Q1 2027)	HL7 / FHIR Standard Clinical Interoperability	Seamless electronic health record (EHR) exchange with diagnostic imaging centers.
Phase 4 (Q2 2027)	Integrated Razorpay / UPI Payment Gateway	Online contactless consultation fee payments prior to hospital arrival.

CHAPTER 17: CONCLUSION & BIBLIOGRAPHY

17.1 Project Conclusion

The development of **MediCare AI: Smart Hospital & Doctor Appointment Management System** successfully fulfills all academic requirements outlined by the **Central Board of Secondary Education (CBSE)** for the Class 12 Computer Science Investigatory Project. Through systematic application of Object-Oriented Programming (OOP) in Python, relational schema design in SQLite3, and modern GUI/Web interface paradigms, the project proves that robust, enterprise-grade healthcare management tools can be constructed with zero proprietary software dependencies.

Working as an investigatory software developer from **PM SHRI Kendriya Vidyalaya ASC Centre** alongside my project partner **AMAN RAJ**, we gained invaluable practical experience in relational schema design, collision-prevention algorithms, cryptographic security, WSGI cloud routing, and automated document compilation. MediCare AI stands as a functional, battle-tested software solution ready to optimize clinical workflows and advance the digitization of Indian healthcare.

17.2 References & Bibliography

1. **Computer Science with Python (Class XII)** by Sumita Arora, Dhanpat Rai & Co. Publications, New Delhi.
2. **Computer Science with Python - Textbook for Class XII**, National Council of Educational Research and Training (NCERT), New Delhi.
3. **Python Official Documentation (v3.12)**: Standard Library, `sqlite3`, `json`, `datetime` modules — <https://docs.python.org/3/>
4. **CustomTkinter Documentation** by Tom Schimansky: Modern UI elements for Python — <https://customtkinter.tomschimansky.com/>
5. **Flask Documentation (v3.0.x)**: The Pallets Projects, Web Development Microframework — <https://flask.palletsprojects.com/>
6. **ReportLab Reference Manual**: Dynamic PDF Generation with Python — <https://www.reportlab.com/docs/reportlab-userguide.pdf>
7. **SQLite SQL Database Engine Documentation**: ACID Transactions and Foreign Key Pragmas — <https://www.sqlite.org/docs.html>
8. **CBSE Curriculum & Assessment Guidelines** for Senior Secondary Computer Science (Code 083) — <https://cbseacademic.nic.in/>

PROJECT INVESTIGATOR & SOFTWARE DEVELOPER

DIVYANSHU KUMAR (Class Roll No: **12115**) | In Collaboration with **AMAN RAJ** (Class Roll No: **12110**)
Class XII A (Science Stream) • PM SHRI KENDRIYA VIDYALAYA ASC CENTRE, BENGALURU